Refined Step A Target Selector for the Polymer Safe-MPC Filter

Internal Technical Report

March 29, 2026

Abstract

This report documents the current steady-state target selector used by the repository's safe-MPC / Lyapunov-filter path after removal of the earlier multi-mode selector stack. The selector is formulated as a single convex steady-state program in scaled deviation coordinates. It keeps the disturbance target fixed at the current observer estimate, penalizes output-reference mismatch softly, and adds explicit anchoring terms that tie the steady target to the actual operating region. The purpose of the selector is not to replace the closed-loop tracking objective. Its purpose is to produce a feasible and numerically well-centered steady package $(x_s, u_s, d_s, y_s, r_s)$ that can be used consistently by the safety filter, diagnostics, and optional admissible-target experiments. This report records the exact optimization problem, weight construction, implementation choices, diagnostics, and practical implications for the current codebase.

1 Introduction

The safe-MPC filter used in this repository has two logically distinct tasks:

1. generate a closed-loop control candidate, typically from offset-free MPC or RL,
2. assess and, if needed, correct that candidate relative to a Lyapunov-centered steady package.

The second task requires a meaningful equilibrium target. Earlier selector variants often returned output-side targets that were numerically feasible but poorly centered for the downstream Lyapunov test. The main symptoms observed during debugging were:

- x_s remained unrealistically close to zero in scaled deviation coordinates,
- u_s stayed too close to nominal or too insensitive to the current operating region,
- previous-target smoothing preserved continuity but did not actively move the selector away from a poor center,
- the admissible output target could appear reasonable even when the associated steady package was a poor center for the QCQP correction layer.

The refined Step A selector addresses this by solving one steady-state problem with three explicit centering mechanisms:

a) an anchor toward the currently applied input,

b) temporal smoothing toward the previous target,

c) a weak anchor toward the current state estimate.

2 Augmented Offset-Free Model and Coordinates

The repository uses an augmented offset-free model

$$z_{k+1} = A_{\text{aug}} z_k + B_{\text{aug}} u_k, \tag{1}$$

$$y_k = C_{\text{aug}} z_k, \tag{2}$$

with

$$z_k = \begin{bmatrix} x_k \\ d_k \end{bmatrix}, \qquad x_k \in \mathbb{R}^{n_x}, \quad d_k \in \mathbb{R}^{n_d}, \quad u_k \in \mathbb{R}^{n_u}, \quad y_k \in \mathbb{R}^{n_y}. \tag{3}$$

In the active implementation, the augmented state is ordered as

$$z_k = [x_k; d_k], \tag{4}$$

and the selector assumes the disturbance block occupies the final n_d coordinates of z_k. The extracted matrices are

$$A = A_{\text{aug}}(1{:}n_x, 1{:}n_x), \tag{5}$$

$$B_d = A_{\text{aug}}(1{:}n_x, n_x{+}1{:}n_x{+}n_d), \tag{6}$$

$$B = B_{\text{aug}}(1{:}n_x, :), \tag{7}$$

$$C = C_{\text{aug}}(:, 1{:}n_x), \tag{8}$$

$$C_d = C_{\text{aug}}(:, n_x{+}1{:}n_x{+}n_d). \tag{9}$$

All quantities entering the selector are expected to be in the repository's scaled deviation coordinates. This is essential: the selector, the downstream safety filter, and the debug diagnostics all assume a common coordinate system. Mixing physical units, min-max scaled quantities, and deviation coordinates is one of the most common failure modes in this codebase.

3 Runtime Data Passed to the Selector

At control step k, the selector receives:

$$\hat{z}_k = \begin{bmatrix} \hat{x}_k \\ \hat{d}_k \end{bmatrix}, \tag{10}$$

$$y_{\text{sp},k} \in \mathbb{R}^{n_r}, \tag{11}$$

$$u_{\text{applied},k} \in \mathbb{R}^{n_u}, \tag{12}$$

where $n_r = n_y$ when no controlled-output map is used and n_r may be smaller when a row selector H is applied.

It may also receive the previous valid target

$$(x_s^{\text{prev}}, u_s^{\text{prev}}), \tag{13}$$

which activates the temporal smoothing terms.

In the current design, the disturbance target is not optimized. It is frozen at the current estimate:

$$d_s = \hat{d}_k. \tag{14}$$

This is a deliberate design choice. The present selector aims to improve the *centering* of the steady package without introducing a second layer of disturbance optimization.

4 Controlled Outputs and Reference Variable

The selector supports an optional controlled-output map

$$r_s = Hy_s, \tag{15}$$

with

$$y_s = Cx_s + C_d d_s. \tag{16}$$

If no matrix H is provided, the implementation uses

$$H = I, \qquad r_s = y_s. \tag{17}$$

The repository currently keeps raw scheduled setpoint tracking as the downstream control objective by default. Therefore:

- the selector computes y_s and r_s for centering and diagnostics,
- the MPC candidate still tracks raw y_{sp} by default unless the tracking-target policy is intentionally changed.

This distinction matters. The selector is first being used to improve the equilibrium center supplied to the safety layer, not to redefine the entire closed-loop tracking target.

5 Refined Step A Optimization Problem

The refined selector solves for the steady variables (x_s, u_s) and then constructs (d_s, y_s, r_s) from $d_s = \hat{d}_k$ and the model equations.

5.1 Steady-state constraints

The steady-state condition is

$$x_s = Ax_s + Bu_s + B_d d_s. \tag{18}$$

Input bounds are enforced as

$$u_{\min} + u_{\text{tight}} \le u_s \le u_{\max} - u_{\text{tight}}. \tag{19}$$

When enabled, output bounds are also enforced:

$$y_{\min} + y_{\text{tight}} \le y_s \le y_{\max} - y_{\text{tight}}. \tag{20}$$

5.2 Objective

The selector solves the convex quadratic program

$$\begin{aligned} \min_{x_s, u_s} \quad & \|r_s - y_{\text{sp},k}\|_{Q_r}^2 + \|u_s - u_{\text{applied},k}\|_{R_{u,\text{ref}}}^2 \\ & + \|u_s - u_s^{\text{prev}}\|_{R_{\Delta u,\text{sel}}}^2 + \|x_s - x_s^{\text{prev}}\|_{Q_{\Delta x}}^2 \\ & + \|x_s - \hat{x}_k\|_{Q_{x,\text{ref}}}^2 \end{aligned} \tag{21}$$

subject to (18), (19), and optionally (20).

If no previous target exists, the terms

$$\|u_s - u_s^{\text{prev}}\|_{R_{\Delta u,\text{sel}}}^2, \qquad \|x_s - x_s^{\text{prev}}\|_{Q_{\Delta x}}^2 \tag{22}$$

are disabled automatically.

5.3 Interpretation of each term

The output term

$$\|r_s - y_{\mathrm{sp},k}\|_{Q_r}^2 \tag{23}$$

keeps the steady target close to the requested reference.

The applied-input anchor

$$\|u_s - u_{\mathrm{applied},k}\|_{R_{u,\mathrm{ref}}}^2 \tag{24}$$

is the main mechanism that prevents the selector from returning a steady input that remains too close to nominal while the plant is operating elsewhere.

The previous-input and previous-state smoothing terms

$$\|u_s - u_s^{\mathrm{prev}}\|_{R_{\Delta u,\mathrm{sel}}}^2, \qquad \|x_s - x_s^{\mathrm{prev}}\|_{Q_{\Delta x}}^2 \tag{25}$$

regularize temporal motion. They suppress chattering but, by themselves, they do not correct a bad center.

The weak state anchor

$$\|x_s - \hat{x}_k\|_{Q_{x,\mathrm{ref}}}^2 \tag{26}$$

is intentionally the weakest state-side term. The observer state is not a steady quantity, but it still provides directional information after operating-point changes.

6 Weight Construction

The implementation exposes diagonal specifications and alpha multipliers rather than asking the caller to provide every matrix directly.

6.1 Output weight

The reference-tracking weight is

$$Q_r \succeq 0, \tag{27}$$

constructed from `Qr_diag`. If a controlled-output map H is used, the code allows either a diagonal specified directly in the reduced reference dimension or a full-output diagonal that is projected through H.

6.2 Input-side weights

The selector reuses the MPC move-penalty structure as a base matrix

$$R_{\Delta u}^{\mathrm{mpc}}. \tag{28}$$

Then

$$R_{u,\mathrm{ref}} = \alpha_{u,\mathrm{ref}} R_{\Delta u}^{\mathrm{mpc}}, \tag{29}$$

$$R_{\Delta u,\mathrm{sel}} = \alpha_{du,\mathrm{sel}} R_{\Delta u}^{\mathrm{mpc}}, \tag{30}$$

unless direct diagonals `R_u_ref_diag` or `R_delta_u_sel_diag` are supplied.

6.3 State-side weights

The base state weight is chosen by

$$Q_x^{\text{base}} = \begin{cases} C_r^\top Q_r C_r, & \text{if } \texttt{x_weight_base} = \texttt{CtQC}, \\ I, & \text{if } \texttt{x_weight_base} = \texttt{identity}, \end{cases} \tag{31}$$

where

$$C_r = \begin{cases} C, & \text{if } H \text{ is absent}, \\ HC, & \text{if } H \text{ is used}. \end{cases} \tag{32}$$

Then

$$Q_{\Delta x} = \alpha_{dx,\text{sel}} Q_x^{\text{base}}, \tag{33}$$

$$Q_{x,\text{ref}} = \alpha_{x,\text{ref}} Q_x^{\text{base}}, \tag{34}$$

unless direct diagonals `Q_delta_x_diag` or `Q_x_ref_diag` are provided.

6.4 Default hierarchy

The intended qualitative ordering is

$$Q_r \gg R_{u,\text{ref}} \sim R_{\Delta u,\text{sel}} > Q_{\Delta x} > Q_{x,\text{ref}}. \tag{35}$$

The code defaults are:

$$\alpha_{u,\text{ref}} = 0.5, \tag{36}$$

$$\alpha_{du,\text{sel}} = 0.5, \tag{37}$$

$$\alpha_{dx,\text{sel}} = 0.05, \tag{38}$$

$$\alpha_{x,\text{ref}} = 0.01. \tag{39}$$

These values are not claimed to be optimal. They encode the intended design priority:

1. output quality must dominate,
2. the steady input should stay near the actual operating region,
3. temporal smoothness should regularize target motion,
4. the transient state anchor should remain weak.

7 Convexity and Numerical Properties

The refined Step A problem is a convex quadratic program under the current implementation assumptions:

- all objective matrices are diagonal or symmetric positive semidefinite,
- the steady-state equations are affine,
- input and optional output bounds are affine.

The problem is therefore well suited for CVXPY with standard conic/QP backends. The code attempts a configured solver sequence and accepts statuses in

$$\{\texttt{optimal}, \texttt{optimal_inaccurate}\}. \tag{40}$$

Actual solver warm start is enabled by:

a) setting variable initial values from the previous target when available,

b) calling the CVXPY solve with `warm_start=True`.

This is important because earlier code paths carried previous-target information without using it as a true solver warm start.

8 Returned Steady Package and Diagnostics

On success, the selector returns the package

$$\{x_s, u_s, d_s, y_s, r_s\} \tag{41}$$

along with metadata:

$$\{\texttt{success}, \texttt{status}, \texttt{solver}, \texttt{objective_value}, \texttt{objective_terms}, \ldots\}. \tag{42}$$

The term-by-term breakdown includes:

$$\texttt{target_tracking},\ \texttt{u_applied_anchor},\ \texttt{u_prev_smoothing},\ \texttt{x_prev_smoothing},\ \texttt{xhat_anchor}. \tag{43}$$

Additional diagnostics include:

- dynamic residual infinity norm,
- bound-violation infinity norm,
- whether previous-target smoothing was active,
- the diagonal weights actually used,
- $\|x_s - \hat{x}_k\|_\infty$,
- $\|x_s - x_s^{\text{prev}}\|_\infty$,
- $\|u_s - u_{\text{applied},k}\|_\infty$,
- $\|u_s - u_s^{\text{prev}}\|_\infty$,
- the output mismatch $\|r_s - y_{\text{sp},k}\|$.

These diagnostics are exported into the safety-debug bundle and used by the plotting layer.

9 Interaction with the Safe-MPC Filter

The selector is consumed downstream as follows.

9.1 Current target vs effective target

The runners call the selector to obtain the current target. If that current solve fails, the safety layer may reuse the last valid target according to the configured backup policy. The code distinguishes:

$$\text{current target source} \in \{\texttt{current}, \texttt{failed}\}, \tag{44}$$

and

$$\text{effective target source} \in \{\texttt{current}, \texttt{last_valid}, \texttt{none}\}. \tag{45}$$

9.2 Role of the selector in the control architecture

The selector does *not* by itself choose the closed-loop reference tracked by the upstream controller. In the current default path:

- the upstream MPC still tracks raw y_{sp},
- the selector supplies $(x_s, u_s, d_s, y_s, r_s)$ as the center for Lyapunov assessment, QCQP correction, and diagnostics,
- the fallback MPC target is selected separately through the tracking-target policy.

This separation is intentional at the current stage of the project. It lets the selector be improved as a *centering mechanism* without forcing an immediate change in the benchmark tracking objective.

9.3 Lyapunov center

The safety filter evaluates candidates relative to the effective target package. The current Lyapunov tolerance convention remains

$$V(e_x^+) \leq \rho V(e_x) + \varepsilon_{\text{lyap}}, \tag{46}$$

where $\varepsilon_{\text{lyap}}$ is treated as a numerical tolerance rather than as a contraction penalty.

10 Why y_s Can Look Suspicious in Practice

One practical observation from debugging is that y_s can appear very close to the plant output even when the state target x_s does not move much. This is not necessarily a bug in the selector objective. It can arise from the disturbance-augmented structure

$$y_s = Cx_s + C_d d_s, \tag{47}$$

especially when the chosen augmentation allows the disturbance channel to carry a large share of the output target.

Since the current selector keeps

$$d_s = \hat{d}_k, \tag{48}$$

the admissible output target can move substantially through the disturbance component. That is why the debug exporter now stores and plots:

$$Cx_s, \qquad C_d d_s, \qquad y_s = Cx_s + C_d d_s. \tag{49}$$

These plots are essential for determining whether the selector is producing a physically meaningful center or merely matching outputs through the disturbance channel.

11 Tuning Implications

The practical tuning order is:

1. tune Q_r so the selector returns acceptable r_s / y_s,
2. tune $\alpha_{u,\mathrm{ref}}$ so u_s stays near the actual operating region after a setpoint change,
3. tune $\alpha_{du,\mathrm{sel}}$ to suppress chattering in u_s,
4. tune $\alpha_{dx,\mathrm{sel}}$ to smooth x_s without freezing it,
5. tune $\alpha_{x,\mathrm{ref}}$ last, because this anchor should remain weak.

If x_s still remains unrealistically close to zero, the most likely causes are:

- Q_r is dominating the state-side penalties too strongly,
- $Q_x^{\mathrm{base}} = C_r^\top Q_r C_r$ is poorly scaled in directions weakly seen through C_r,
- $\alpha_{x,\mathrm{ref}}$ is too small to move the selector away from a bad center,
- the output target is being explained mainly through the disturbance component $C_d d_s$.

12 What the Current Selector Solves and What It Does Not Solve

The refined Step A selector solves the following problem:

> Find a feasible steady-state target that remains close to the requested output while staying near the actual operating region and evolving smoothly over time.

It does *not* currently solve:

1. free optimization of the disturbance target d_s,
2. explicit robust target calculation under uncertainty sets,
3. a joint redesign of the downstream tracking objective,
4. a guarantee that the returned target is the best possible Lyapunov center in every operating region.

Those remain future research directions. The current selector is deliberately narrower and easier to diagnose.

13 Code Mapping

The canonical implementation is in

- `Lyapunov/target_selector.py`,
- `Simulation/run_mpc_lyapunov.py`,
- `Simulation/run_rl_lyapunov.py`,

- `Lyapunov/safety_filter.py`,
- `Lyapunov/safety_debug.py`.

The key runtime flow is:

1. build the selector configuration with `build_target_selector_config(...)`,
2. solve the refined Step A selector through `prepare_filter_target(...)`,
3. form the effective target by reusing the last valid target if necessary,
4. evaluate or correct the candidate input relative to that effective target inside the safety filter,
5. log the returned package and objective terms to the debug exporter.

14 Conclusion

The refined Step A selector is the repository's current canonical steady-state target calculation for the safe-MPC filter. Relative to the earlier selector family, it is intentionally simpler in architecture but richer in centering information. The central design choice is to keep the disturbance target fixed while improving the quality of the returned equilibrium center through the combined use of output tracking, applied-input anchoring, temporal smoothing, and weak state anchoring.

This design is not presented as a final answer to admissible-target calculation for the polymer system. It is presented as a cleaner and more diagnosable baseline: one convex selector, one clear objective, one stable output schema, and one consistent place from which future selector refinements can proceed.

Reference Context

This report should be read together with the repository's existing offset-free MPC notes and the standard target-calculation perspective used in the MPC literature, especially the material already cited elsewhere in the project on offset-free MPC and steady-state target calculation.